PROGRAMMING & CS FUNDAMENTALS

Design Patterns III: Behavioral Visual Guide

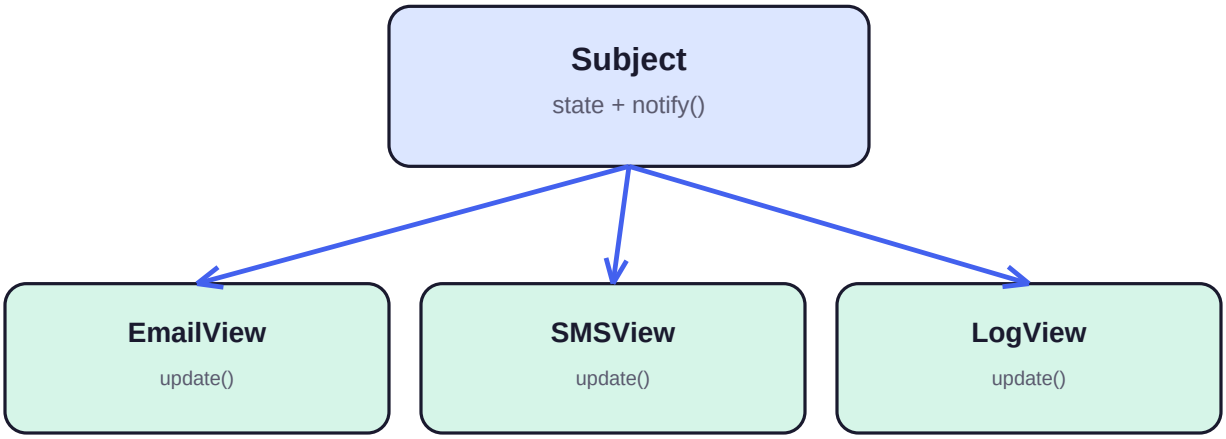
How objects talk —
five ways to organize communication.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Observer: Publish / Subscribe

One subject, many dependents — notify, don't poll.



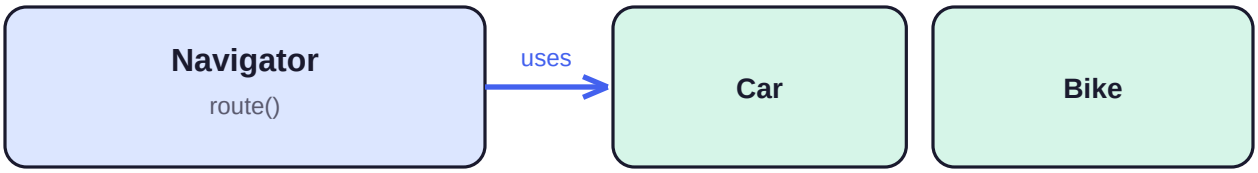

```
subject.attach(email_view); subject.attach(sms_view)
subject.set_state('shipped')   # all observers update()
```

- Subject knows only the OBSERVER interface — observers stay decoupled.
- Event buses, reactive UI, model-view: the observer is everywhere.
- Lapsed-listener leak: observers that never detach pin memory — unsubscribe!

STEP 2/6

Strategy: Swappable Algorithms

Composition's favorite child — the algorithm becomes a part.




```
nav = Navigator(strategy=CarStrategy())   # pick at RUNTIME
nav.route(a, b)
nav.strategy = BikeStrategy()             # swap mid-trip
```

- Kills the if-transport == 'car' chains — new strategy = new class.
- Sorting comparators, payment methods, compression choices: all strategies.
- Strategy vs State: strategy is CHOSEN by the client; state is DRIVEN by the object.

STEP 3/6

Command: Requests as Objects

Encapsulate a call — then queue it, log it, undo it.

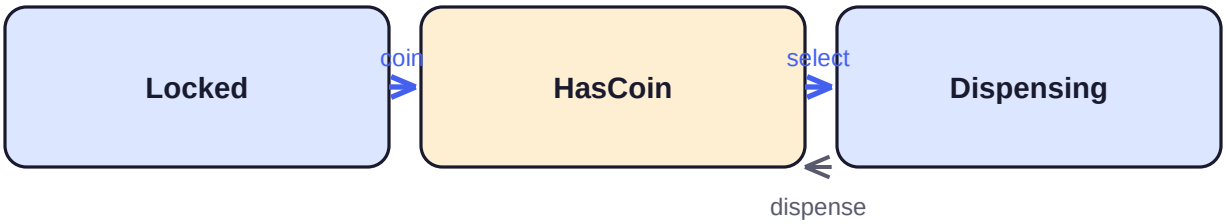
```
class LightOn(Command):
    def execute(self): self.light.on()
    def undo(self): self.light.off()     # undo stack = list of commands
remote.press(LightOn(light))             # invoker never knows 'light'
```

- Undo/redo, macros, job queues, transactional batches — commands make actions data.
- Invoker and receiver never meet: total decoupling of 'who asks' from 'who does'.
- Store the inverse (or a snapshot) per command and undo becomes trivial.

STEP 4/6

State: Behavior Follows State

An object that rewrites its own responses as it changes.




```
class Vending:                        # delegates to current state
    def insert_coin(self): self.state.insert_coin(self)
class HasCoin(State):
    def insert_coin(self, ctx): return 'already paid!'  # per-state logic
```

- Replaces sprawling if-state == X chains — each state is a class with its own rules.
- Transitions live in the states: HasCoin.select() moves the context to Dispensing.
- TCP sockets, game characters, parsers, workflows — state machines in disguise.

STEP 5/6

Template Method: Skeleton with Hooks

The algorithm's SHAPE is fixed; steps are overridden.

```
class DataMiner:                        # the TEMPLATE
    def mine(self):                     # final: do not override
        raw = self.extract()            # hook 1
        clean = self.transform(raw)     # hook 2
        self.load(clean)                # hook 3
class CSVMiner(DataMiner):
    def extract(self): return read_csv(...)   # fill the hooks
```

- Hollywood principle: 'don't call us, we'll call you' — the base drives, subclasses fill in.
- Hooks with default (do-nothing) implementations make steps optional.
- Frameworks are template methods at scale: you fill hooks, the framework runs the show.
- Prefer composition + strategy when the skeleton itself might need to vary.

STEP 6/6

Behavioral Chooser

Which conversation pattern for which pain?

Observer

One-to-many notifications — subject notifies dependents

Strategy

Swappable algorithms — client picks the variant

Command

Requests as data (undo/queue) — execute() + undo()

State

Behavior depends on state — object rewrites its responses

Template Method

Fixed skeleton, varying steps — base drives, hooks vary

Strategy vs State vs Template: strategy varies the WHOLE algorithm by client choice; state varies behavior by internal condition; template fixes the skeleton and varies steps.

STEP ✓/✓

Key Takeaways

Behavioral Patterns — Remember

- Observer: subject notifies dependents via update() — remember to detach (lapsed listeners leak).
- Strategy: swappable algorithm objects, chosen at run time — kills if-chains.
- Command: requests as objects with execute()/undo() — undo stacks, queues, macros.
- State: each state is a class; transitions move the context — replaces state if-chains.
- Template Method: fixed skeleton, overridable hooks — the Hollywood principle.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev